

The ROS programming framework

Fundamentals of Robotics
DISI - UNIVERSITY OF TRENTO
TEACHER: Luigi Palopoli

What is a robot?

- Go back to our Longman definition of a robot

A machine that can move and do some of the work of a person
and is usually controlled by a computer

- Robots are computer controlled machine that operate in the physical world
- How are they programmed?

The world of industrial robots

- Classic industrial robots are quite “closed” environment that execute repetitive tasks
- Robot programming can be done in two ways
 - Guiding: the robot arm is guided (manually or by a remote controller) through a set of points
 - Offline: the robot follows a program written in a script language
- The script languages used for programming are often proprietary
 - But at the end of the day they are extremely simple and comparable

Example – Offline programming

- Consider the following example (human readable pseudo code)

```
Move to P1 (a general safe position)
Move to P2 (an approach to P3)
Move to P3 (a position to pick the object)
Close gripper
Move to P4 (an approach to P5)
Move to P5 (a position to place the object)
Open gripper
Move to P1 and finish
```

Example – Offline programming

- Translation into VAL (Unimate)
- RC+ (EPSON) with vacuum gripper

```
PROGRAM PICKPLACE
  1. MOVE P1
  2. MOVE P2
  3. MOVE P3
  4. CLOSEI 0.00
  5. MOVE P4
  6. MOVE P5
  7. OPENI 0.00
  8. MOVE P1
.END
```

```
Function PickPlace
  Jump P1
  Jump P2
  Jump P3
  On vacuum
  Wait .1
  Jump P4
  Jump P5
  Off vacuum
  Wait .1
  Jump P1
Fend
```

Offline languages

- Each vendor has its own scripting language

Robot brand	Language name
ABB	RAPID
Comau	PDL2
Fanuc	Karel
Kawasaki	AS
Kuka	KRL
Stäubli	VAL3
Yaskawa	Inform

- The scripts are interpreted and translated on the fly into real—time actions

Modern Robots

- Modern robots are much more complex
 - They operate in open environment
 - They need sophisticated perception abilities
 - They have to re-plan in real-time in order to reach to unexpected conditions
 - They interact with humans
 - They collaborate with other robots
- All these requirements transform robot programming into a multidisciplinary activity
- Each discipline has its own programming framework and languages.
- So integration can be a titanic effort

The babel tower

- Each community has its own mindset and language



Bruegel, 1563

- Will it stand or will it collapse?

A quick (and rather incomplete) survey

Activity	Methodologies	Framwork → Languages
Sensing and actuation	Micro-controller programming	FreeRTOS, Proprietary IDE → C
Kinematic and Dynamic Control	Model based control design	MATLAB-Simulink → C/C++
Perception (Detection/classification)	Machine Learning	Yolo, OpenCV → Python, C/C++
SLAM, Data fusion	Statistical learning	Matlab-Simulink → C/C++
Motion Planning	Optimisation techniques	Matlab-Simulink, Libraries → C++
Task Planning	Discrete optimisation, formal methods	PDDL,... → Python, C++

Additional problems

- Many applications require robot-to-robot communication
- Sometimes we have to use external services in the cloud (e.g., for strategic decisions)
- We operate with multiple types of hardware
 - Microcontrollers
 - GPU
 - Industrial PC
- The different computations have timing constraints
 - Particularly true for unstable systems like drones or legged robots
- Finally, most of the times the developers of SW components for robotics are not necessarily computer experts
 - ...not your case



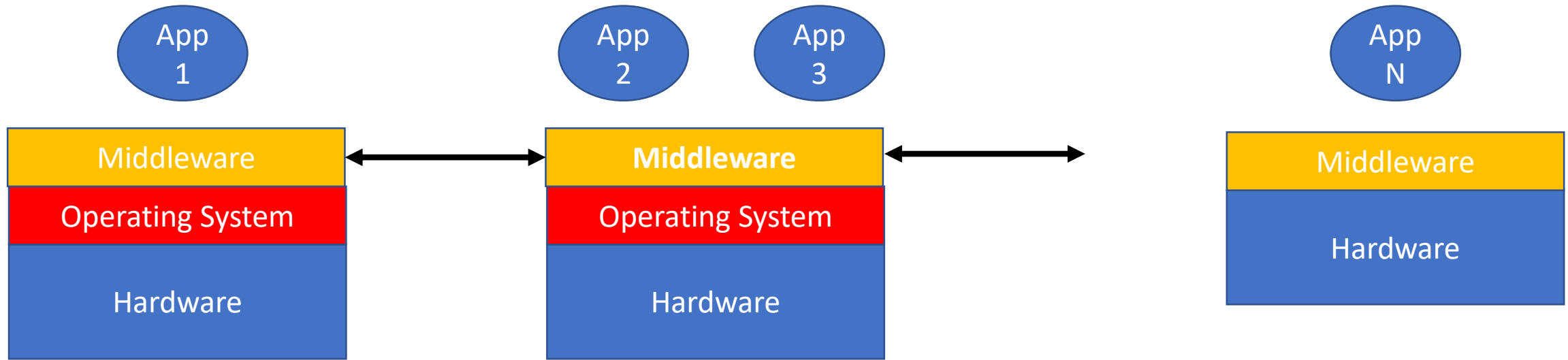
The solution

- The solution is the adoption of a middleware

Middleware: computer software that enables communication between multiple software applications, possibly running on more than one machine.

- Middleware enable the development of distributed, multilingual applications without requiring the direct use of Operating System and networking primitives

The concept of middleware



- Applications communicate through standard messages
- They are unaware of where the partner application is located (it can be on the same node or anywhere)
- The middleware sees to the correct delivery of the messages
 - E.g., by implementing a publish-subscribe mechanism
- Applications can be written in any language as long as they get connected through a client library
- Applications do not rely directly on any OS (which in some case is not there)

The concept of middleware

- In a word a Middleware is an abstraction layer that significantly simplifies the development and the integration of distributed applications
- There are many types of middleware

Type	Services	Examples
Message Oriented Middleware	Receiving and sending of messages over distributed applications	Amazon Simple Notification System (SNS), IBM MQ, Amazon AWS IoT Core
Remote Procedure Call (RPC)	Calling procedures on remote systems and performing synchronous or asynchronous interactions	Oracle: Open Network Computing RPC, SOAP (xmlRPC),...
Database middleware	Allowing for direct access to databases	ODBC, JDBC, EDA/SQL ...
Embedded Middleware	Supporting embedded applications	zMQ, ROS , IoT middlewares....

Middleware for robotics

- **Robot Operating System (ROS)**

- De-facto standard for hundreds of available components
- Some issues with complexity and latency... (addressed by ROS 2.0)

- **zMQ**

- General message oriented middleware for lightweight embedded applications
- Usable (and used) in robotics
- Very low and controlled latencies
- Integrated into ROS 2.0

In this course, we will use ROS for the huge availability of software and services

DIGGING INTO ROS

Historical notes

- ROS was initially developed in Stanford (Artificial Intelligence Laboratory) in 2007
- Since 2013 the project was taken over by OSRF (Open source Robotics Foundations)
- It has become a de-fact standard, extremely popular especially (but not exclusively) at the academic level
- Growing industrial success and recognition

Main features

- **Peer-to-peer:** applications use a standardized API to exchange messages
- **Distributed:** the framework fully supports applications running on multiple computers
- **Multilingual:** the RoS components can be developed in any language as long as a client library exists (C++, Python, Matlab, Java, Ruby...)
- **Light-weight (especially 2.0):** applications are connected through a very simple and thin layer
- **Free and open-source:** most of RoS applications are open-source and free to use. But, its permissive licensing policy allows for the development of closed and commercial applications

ROS Master

- The ROS master is the manager of the middleware
 - Provides naming and registration services
 - It tracks publishers and subscriber to *topics and services*
 - In essence it enables nodes to see each other and establish peer-to-peer communication
 - It provides the Parameter Server (access to a dictionary containing the parameters of all objects)
- It provides an XMLRPC-based API, which client libraries like *roscpp* and *rospy* connect to in order and retrieve information

- The ROS Master activated by

```
> roscore
```

which loads ROS master and other essential components

ROS Nodes

- A node is a single process delivering a service
- It is an executable program that is compiled and executed
- Nodes can be combined together to form graphs
- Example:
 - One node manages the laser range-finder
 - One node implements localisation
 - One node implements motion planning
 - One node implements wheel control
- The use of nodes allows the developers to decouple their work and improve maintainability and robustness of their code

Node Graphs

- All nodes have a graph resource name that uniquely identifies it in the system, e.g.

`/hokuyo_laser`

- Node also have a type (package resource name + name of the executable file)
When requested to activate a node, RoS scans the package looking for
- all nodes with the executable name and chooses the first one

ROS Nodes

- Nodes have to be registered with the master
 - They are organised in packages
- Execution of a node

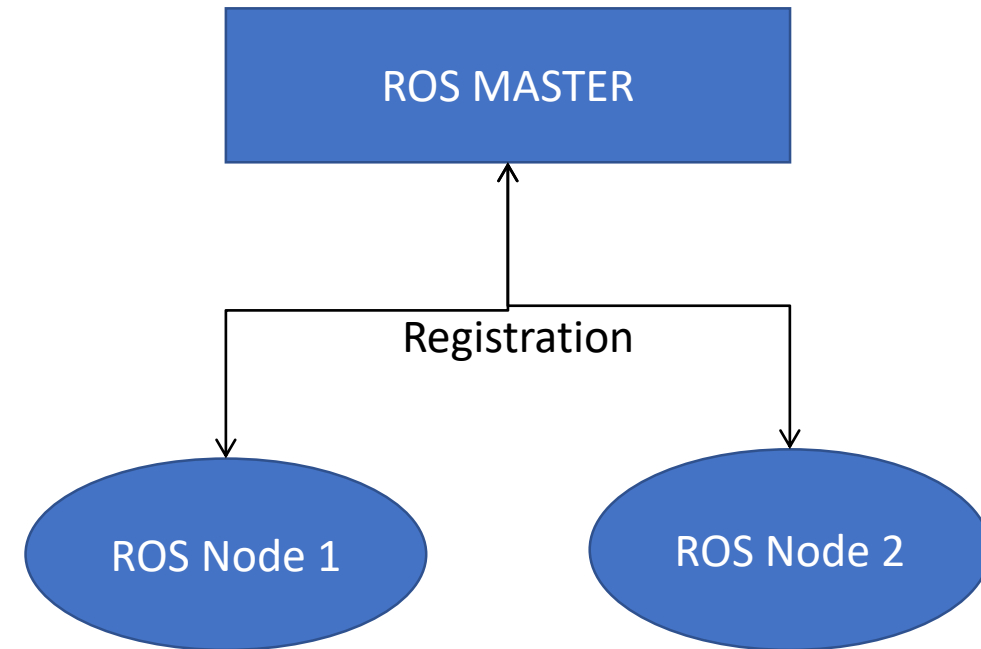
```
roslaunch package_name node_name
```

List of active nodes

```
roslaunch list
```

Information retrieval on a node

```
roslaunch info node_name
```

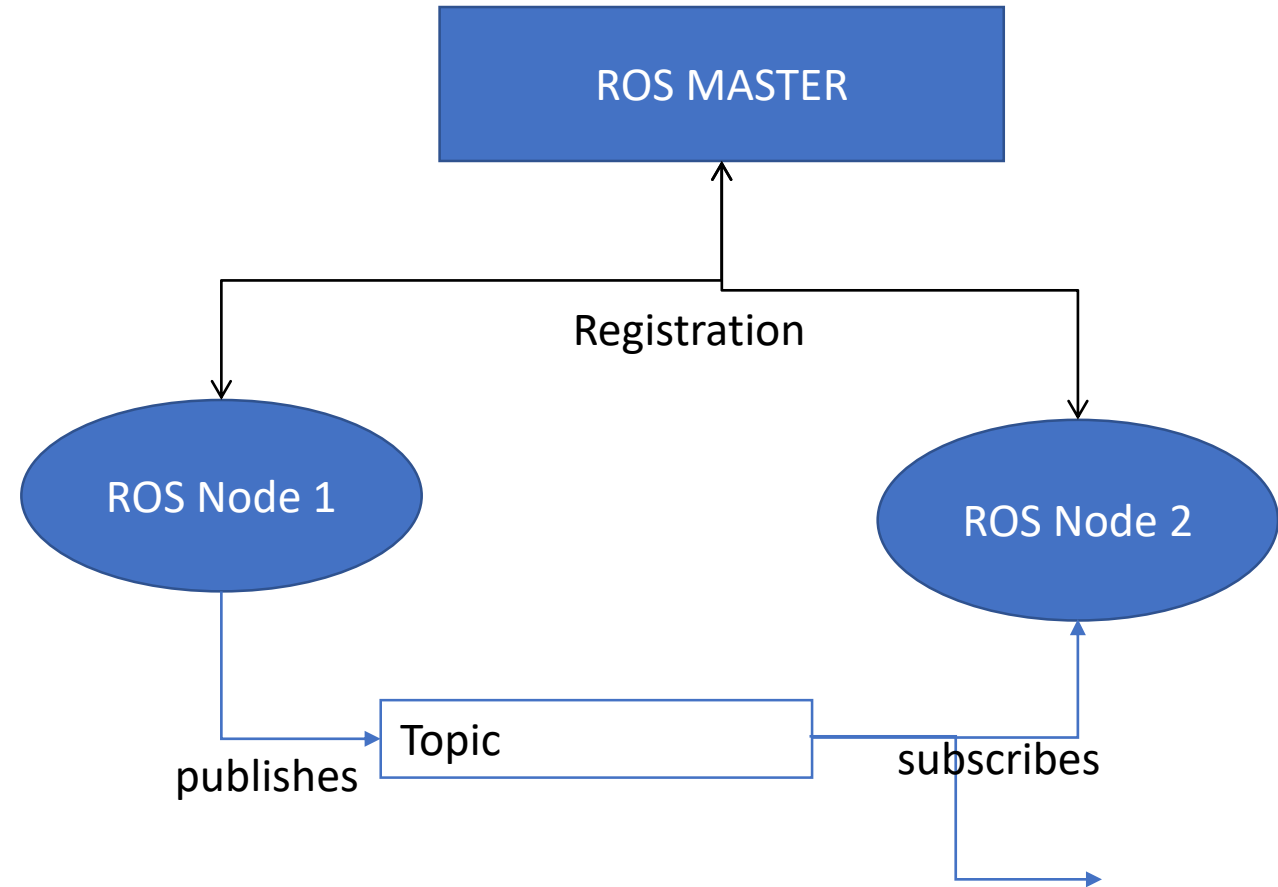


Communication between nodes

- There are three ways for nodes to communicate
 - Streaming topics
 - RPC service
 - Parameter server
- Let us focus on topics and messages

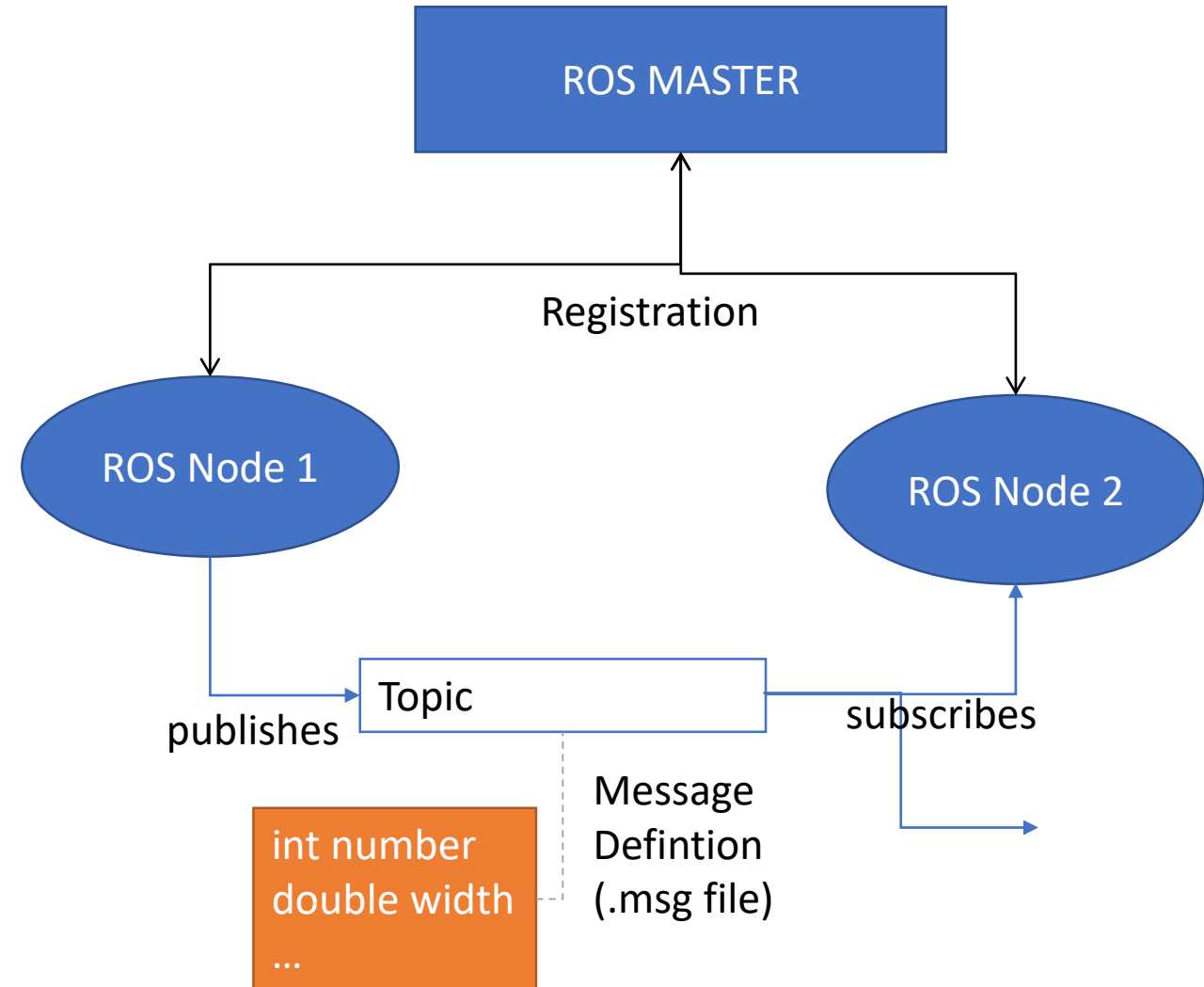
Topics and messages

- A topic is a name for a stream of messages
- Topics are the primary way for establishing a communication
-
- The typical situation is one publisher and multiple subscribers
- This scheme is intended for unidirectional streaming
- The receiver does not need to know the publisher, but only the topic



Messages

- Nodes communicate through topics exchanging *messages*
- A message defines the type of a topic
- It is defined in a *.msg* file
- A message is a data structure comprising typed fields
- Fields
 - Integers
 - Booleans
 - Strings
 - struct (c-like)



Messages

- The type of a topic (i.e., the message structure) can be seen by

```
>rostopic type /topic
```

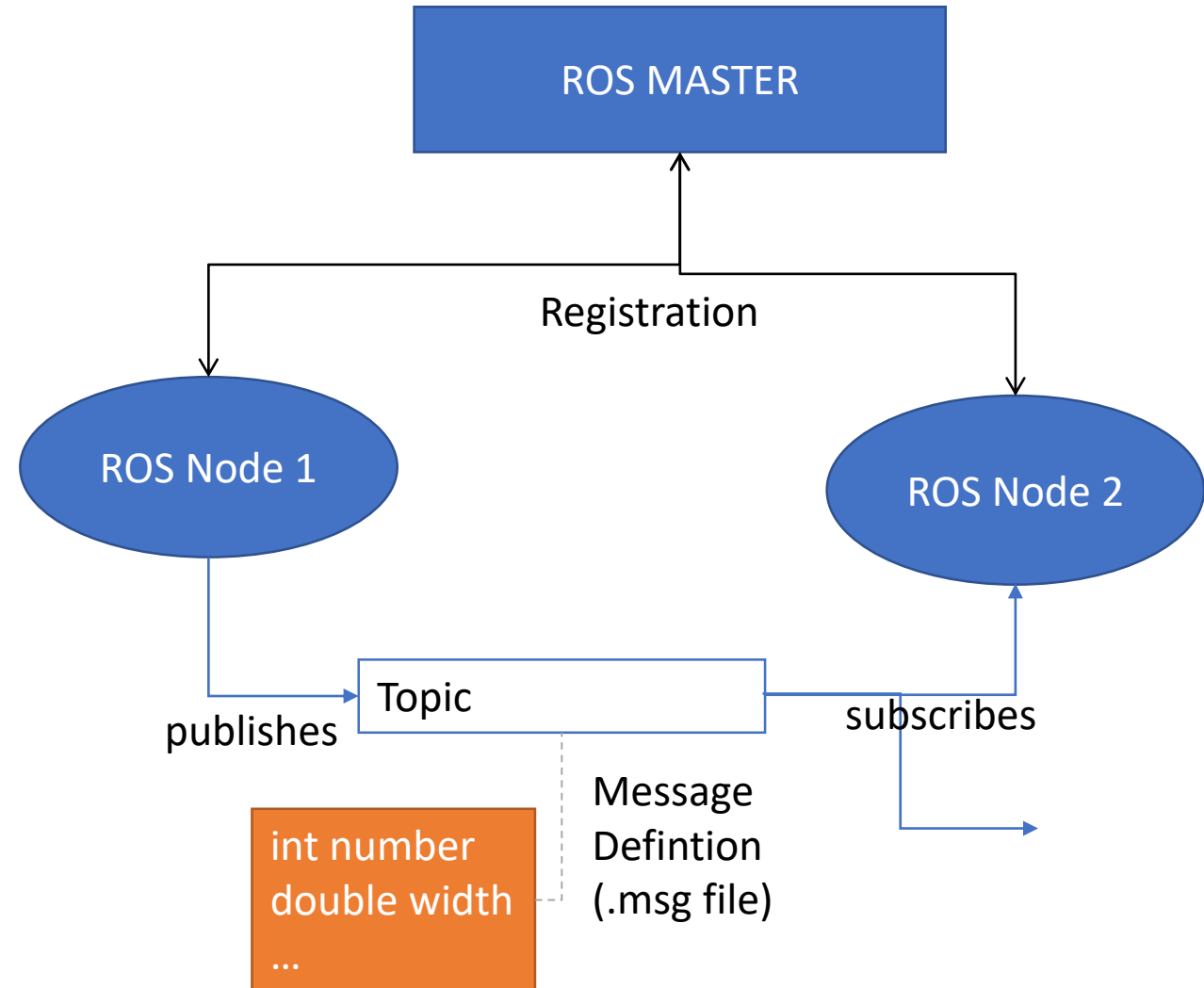
- A message can be published by

```
> rostopic pub /type  
type data
```

- Naming convention:
package+name of the .msg file.

Es.

```
std_msgs/msg/String.msg
```



Message examples

Pose stamped examples

geometry_msgs/Point.msg

```
float64 x  
float64 y  
float64 z
```

sensor_msgs/Image.msg

```
std_msgs/Header header  
  uint32 seq  
  time stamp  
  string frame_id  
uint32 height  
uint32 width  
string encoding  
uint8 is_bigendian  
uint32 step  
uint8[] data
```

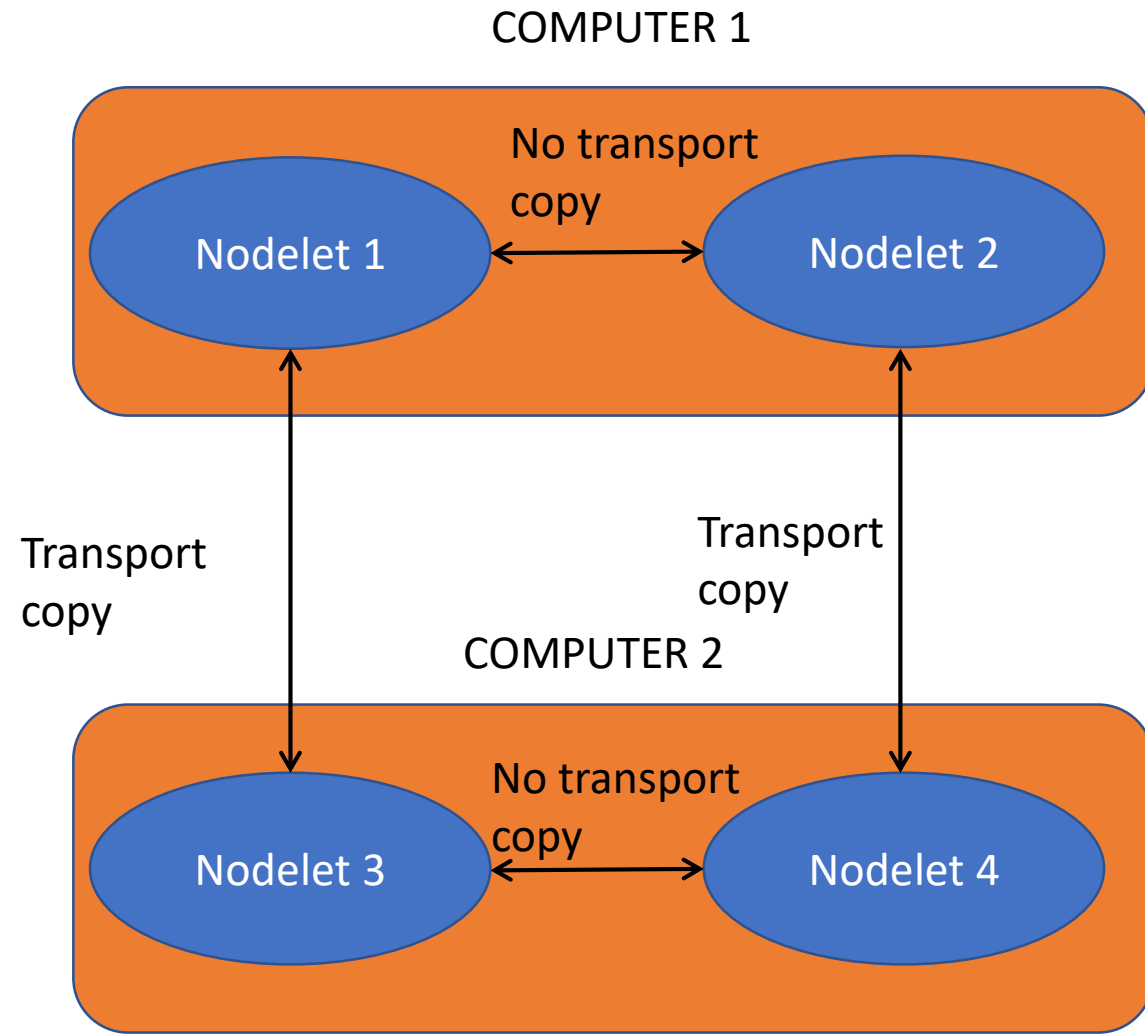
geometry_msgs/PoseStamped.msg

```
std_msgs/Header header  
uint32 seq  
time stamp  
string frame_id  
geometry_msgs/Pose pose  
  geometry_msgs/Point position  
    float64 x  
    float64 y  
    float64 z  
  geometry_msgs/Quaternion orientation  
    float64 x  
    float64 y  
    float64 z  
    float64 w
```



Nodelet

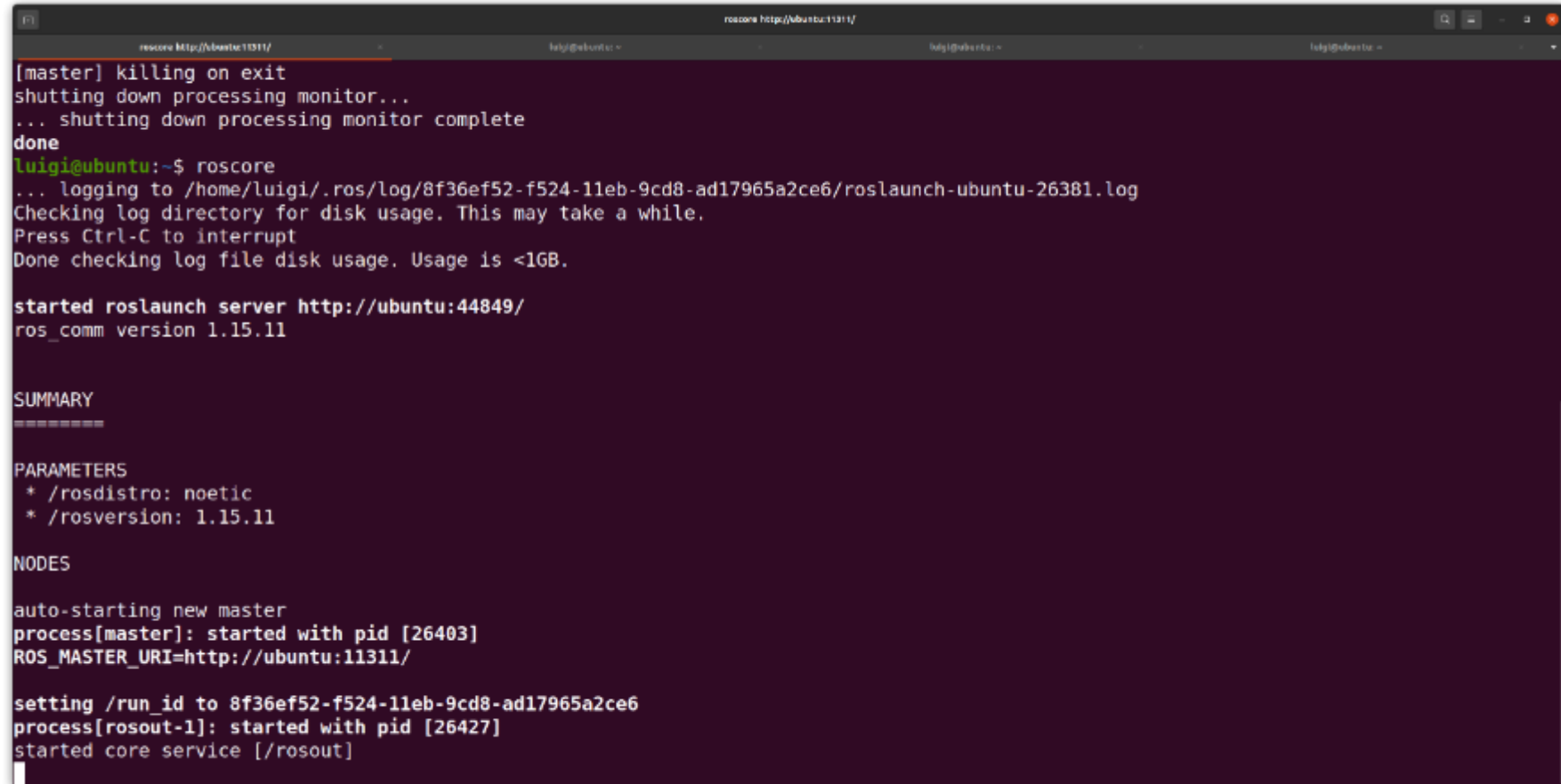
- If we know that two programs execute on the same hardware, the standard communication could be heavy
- The Nodelet package implements a zero transport copy when two nodes are on the same hardware.
- In essence the nodes live in the same process
- This feature is quite advanced



Example

- Let us look at a first example going through the ROS tutorial
- Step 1: open a console and activate the master

```
> roscore
```

A terminal window with a dark purple background and white text. The window title is 'roscore http://ubuntu:11311/'. The output shows the master shutting down, then starting a new roscore process. It logs the path to the log directory, checks disk usage, and starts the roslaunch server. It then displays a summary of parameters (rostdistro: noetic, rosversion: 1.15.11) and nodes (auto-starting new master, process[master] started with pid [26403], ROS_MASTER_URI=http://ubuntu:11311/, setting /run_id to 8f36ef52-f524-11eb-9cd8-ad17965a2ce6, process[rosout-1] started with pid [26427], started core service [/rosout]).

```
roscore http://ubuntu:11311/
[master] killing on exit
shutting down processing monitor...
... shutting down processing monitor complete
done
luigi@ubuntu:~$ roscore
... logging to /home/luigi/.ros/log/8f36ef52-f524-11eb-9cd8-ad17965a2ce6/roslaunch-ubuntu-26381.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://ubuntu:44849/
ros_comm version 1.15.11

SUMMARY
=====

PARAMETERS
* /rostdistro: noetic
* /rosversion: 1.15.11

NODES

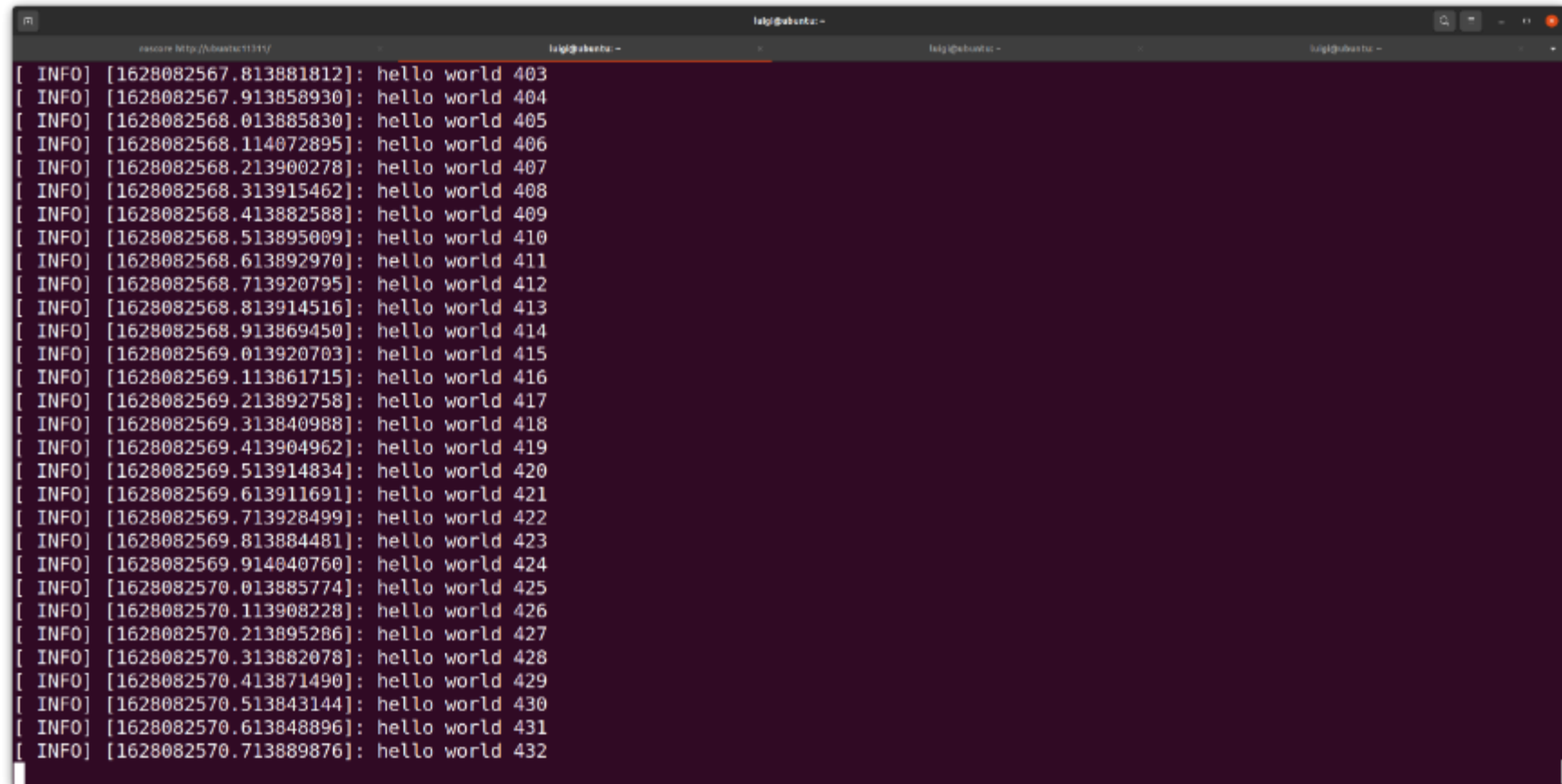
auto-starting new master
process[master]: started with pid [26403]
ROS_MASTER_URI=http://ubuntu:11311/

setting /run_id to 8f36ef52-f524-11eb-9cd8-ad17965a2ce6
process[rosout-1]: started with pid [26427]
started core service [/rosout]
```

Example

- Step 2: start a talker demo

```
> rosrun roscpp_tutorials talker
```



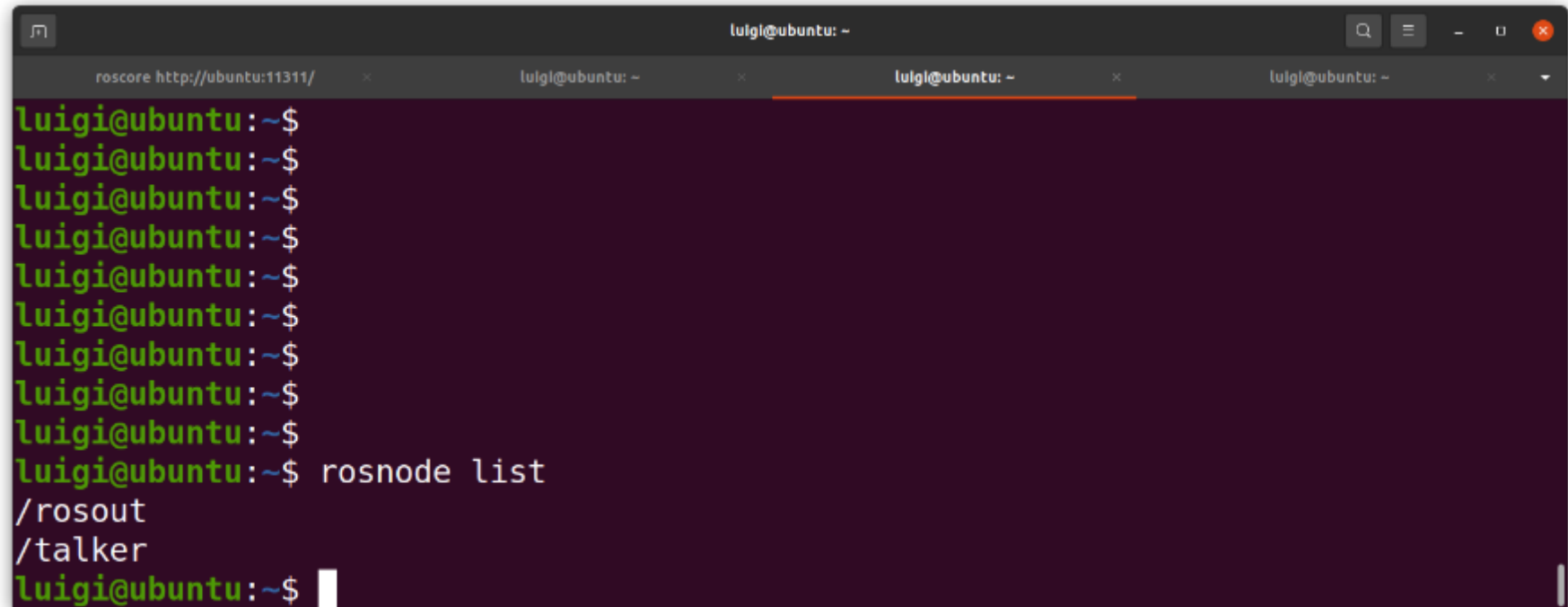
```
[ INFO] [1628082567.813881812]: hello world 403
[ INFO] [1628082567.913858930]: hello world 404
[ INFO] [1628082568.013885830]: hello world 405
[ INFO] [1628082568.114072895]: hello world 406
[ INFO] [1628082568.213900278]: hello world 407
[ INFO] [1628082568.313915462]: hello world 408
[ INFO] [1628082568.413882588]: hello world 409
[ INFO] [1628082568.513895009]: hello world 410
[ INFO] [1628082568.613892970]: hello world 411
[ INFO] [1628082568.713920795]: hello world 412
[ INFO] [1628082568.813914516]: hello world 413
[ INFO] [1628082568.913869450]: hello world 414
[ INFO] [1628082569.013920703]: hello world 415
[ INFO] [1628082569.113861715]: hello world 416
[ INFO] [1628082569.213892758]: hello world 417
[ INFO] [1628082569.313840988]: hello world 418
[ INFO] [1628082569.413904962]: hello world 419
[ INFO] [1628082569.513914834]: hello world 420
[ INFO] [1628082569.613911691]: hello world 421
[ INFO] [1628082569.713928499]: hello world 422
[ INFO] [1628082569.813884481]: hello world 423
[ INFO] [1628082569.914040760]: hello world 424
[ INFO] [1628082570.013885774]: hello world 425
[ INFO] [1628082570.113908228]: hello world 426
[ INFO] [1628082570.213895286]: hello world 427
[ INFO] [1628082570.313882078]: hello world 428
[ INFO] [1628082570.413871490]: hello world 429
[ INFO] [1628082570.513843144]: hello world 430
[ INFO] [1628082570.613848896]: hello world 431
[ INFO] [1628082570.713889876]: hello world 432
```

Example

- Step 3.1: analyse the newly activated node

List of all active nodes

```
> rosnodet list
```

A terminal window titled 'luigi@ubuntu: ~' with four tabs. The first tab is 'roscat http://ubuntu:11311/'. The terminal shows a series of 'luigi@ubuntu:~\$' prompts. The 10th prompt is followed by the command 'roscat list'. The output shows two active nodes: '/roscat' and '/talker'.

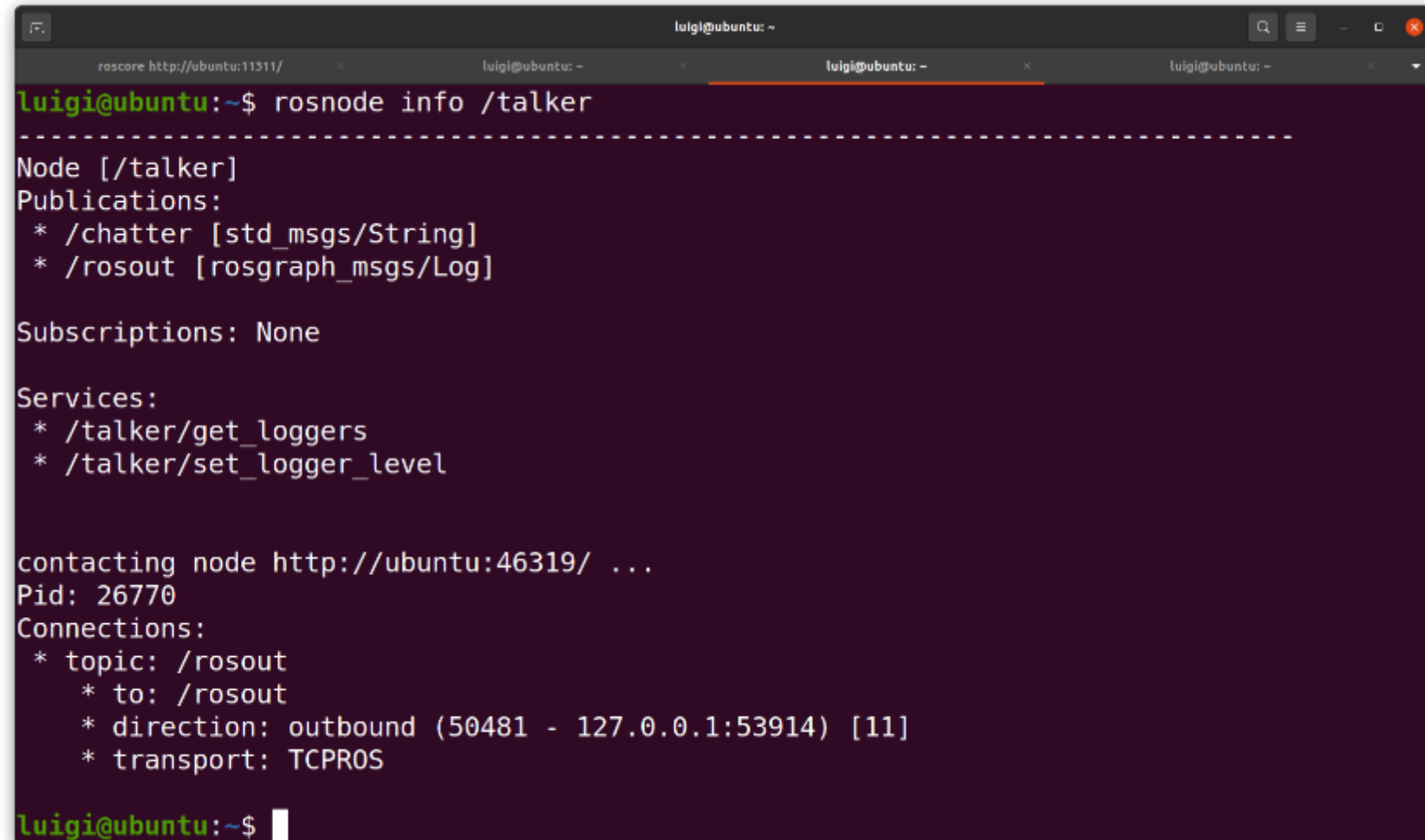
```
luigi@ubuntu:~$  
luigi@ubuntu:~$  
luigi@ubuntu:~$  
luigi@ubuntu:~$  
luigi@ubuntu:~$  
luigi@ubuntu:~$  
luigi@ubuntu:~$  
luigi@ubuntu:~$  
luigi@ubuntu:~$  
luigi@ubuntu:~$ roscat list  
/roscat  
/talker  
luigi@ubuntu:~$
```

Example

- Step 3.2: analyse the newly activated node

Info about talker

```
> rosnode info /talker
```

A terminal window with a dark purple background and white text. The window title is 'luigi@ubuntu: ~'. The prompt is 'luigi@ubuntu:~\$'. The command 'roscall info /talker' has been entered. The output shows a dashed line, followed by 'Node [/talker]', 'Publications:' with two entries: '* /chatter [std_msgs/String]' and '* /rosout [roscpp_msgs/Log]', 'Subscriptions: None', 'Services:' with two entries: '* /talker/get_loggers' and '* /talker/set_logger_level', 'contacting node http://ubuntu:46319/ ...', 'Pid: 26770', 'Connections:' with four entries: '* topic: /rosout', '* to: /rosout', '* direction: outbound (50481 - 127.0.0.1:53914) [11]', and '* transport: TCPROS'. The prompt 'luigi@ubuntu:~\$' is visible at the bottom.

```
luigi@ubuntu:~$ roscall info /talker
-----
Node [/talker]
Publications:
* /chatter [std_msgs/String]
* /rosout [roscpp_msgs/Log]

Subscriptions: None

Services:
* /talker/get_loggers
* /talker/set_logger_level

contacting node http://ubuntu:46319/ ...
Pid: 26770
Connections:
* topic: /rosout
  * to: /rosout
  * direction: outbound (50481 - 127.0.0.1:53914) [11]
  * transport: TCPROS

luigi@ubuntu:~$
```

Example

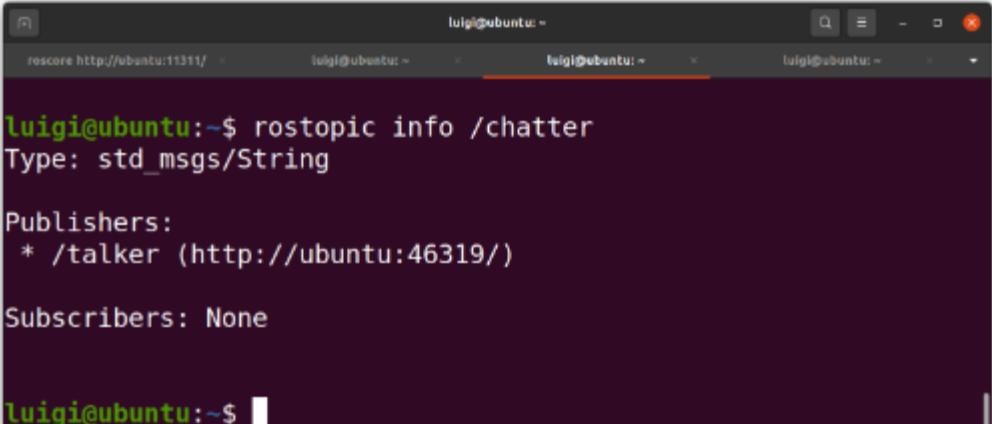
- Step 3.3: analyse the newly activated node

Info about the *chatter* topic

```
> rostopic info /chatter
```

Type check

```
> rostopic type /chatter
```

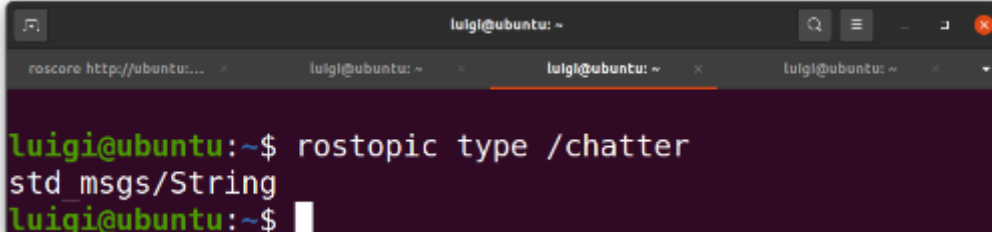
A terminal window with a dark purple background. The prompt is 'luigi@ubuntu:~\$'. The command 'rostopic info /chatter' has been executed. The output shows the topic type as 'std_msgs/String', one publisher at '/talker (http://ubuntu:46319/)', and no subscribers.

```
luigi@ubuntu:~$ rostopic info /chatter
Type: std_msgs/String

Publishers:
 * /talker (http://ubuntu:46319/)

Subscribers: None

luigi@ubuntu:~$
```

A terminal window with a dark purple background. The prompt is 'luigi@ubuntu:~\$'. The command 'rostopic type /chatter' has been executed. The output shows the topic type as 'std_msgs/String'.

```
luigi@ubuntu:~$ rostopic type /chatter
std_msgs/String
luigi@ubuntu:~$
```

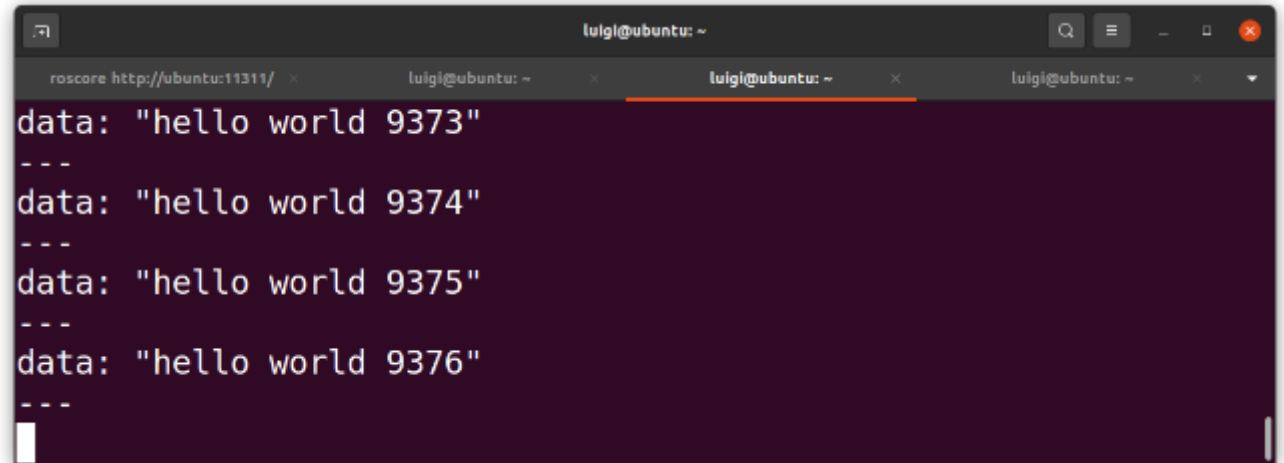
Example

- Step 3.3: analyse the newly activated node

Some more info about the *chatter* topic

Message content

```
rostopic echo /chatter
```

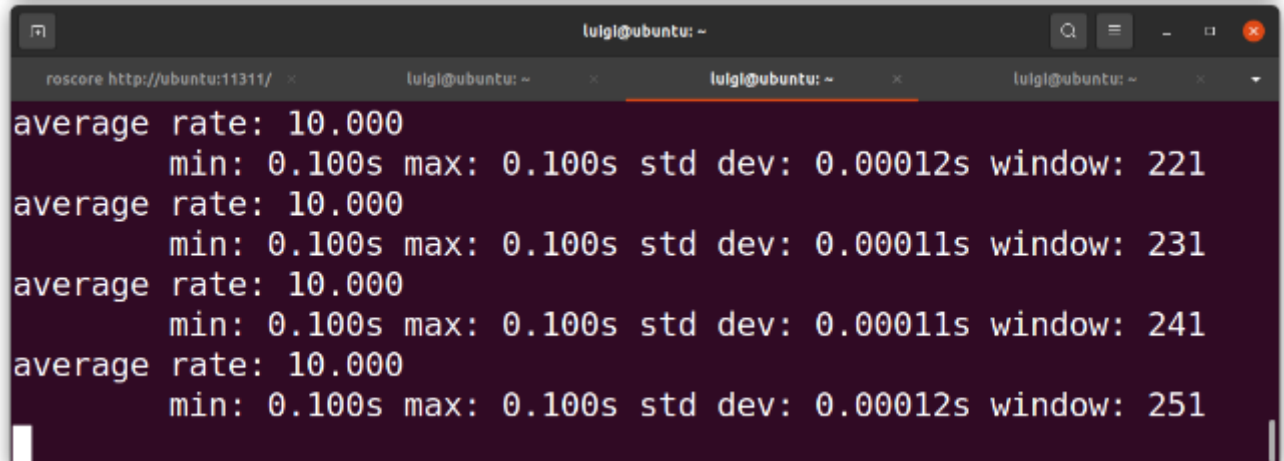


A terminal window titled 'luigi@ubuntu: ~' showing the output of the 'rostopic echo' command for the '/chatter' topic. The output consists of four lines, each starting with 'data:' followed by a string and a sequence number, separated by three dashes '---'. The strings are 'hello world 9373', 'hello world 9374', 'hello world 9375', and 'hello world 9376'.

```
roscore http://ubuntu:11311/ x luigi@ubuntu: ~ x luigi@ubuntu: ~ x luigi@ubuntu: ~  
data: "hello world 9373"  
---  
data: "hello world 9374"  
---  
data: "hello world 9375"  
---  
data: "hello world 9376"  
---
```

Frequency check

```
rostopic hz /chatter
```



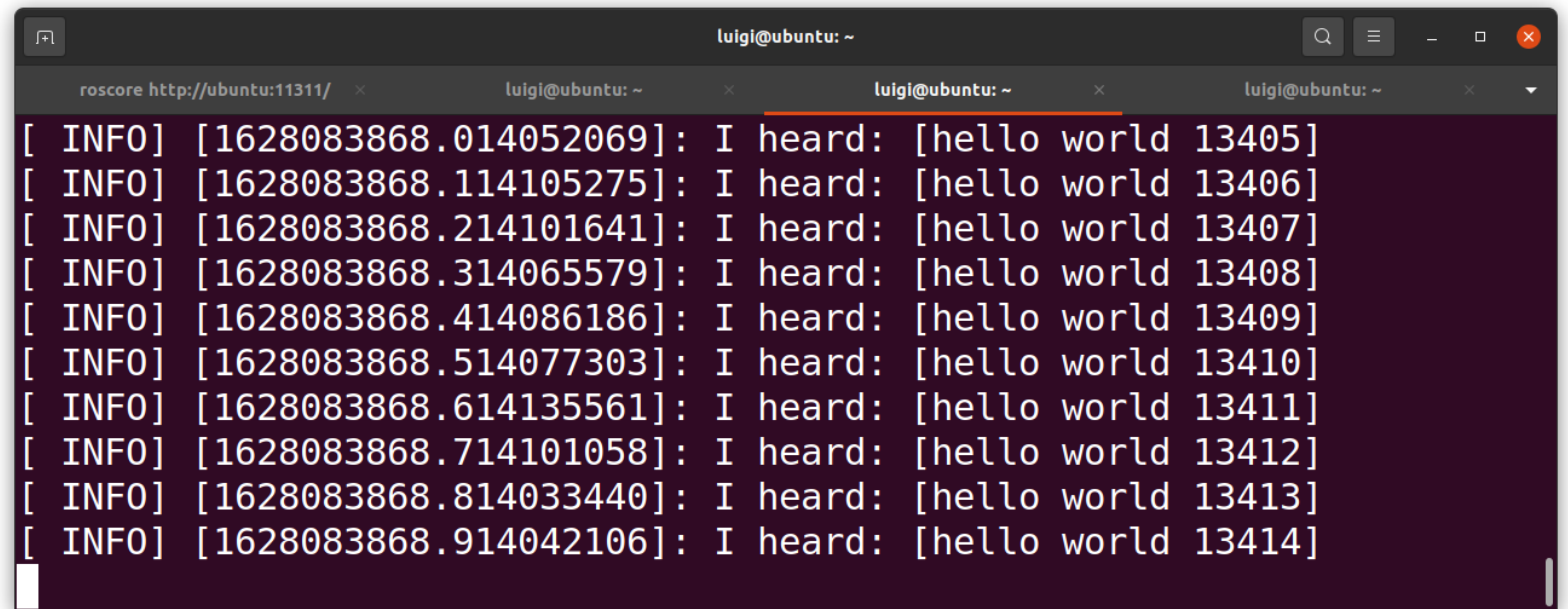
A terminal window titled 'luigi@ubuntu: ~' showing the output of the 'rostopic hz' command for the '/chatter' topic. The output displays four lines of frequency statistics, each starting with 'average rate:' followed by a value, and then 'min:', 'max:', 'std dev:', and 'window:' with their respective values. The average rate is consistently 10.000, and the window values are 221, 231, 241, and 251.

```
roscore http://ubuntu:11311/ x luigi@ubuntu: ~ x luigi@ubuntu: ~ x luigi@ubuntu: ~  
average rate: 10.000  
min: 0.100s max: 0.100s std dev: 0.00012s window: 221  
average rate: 10.000  
min: 0.100s max: 0.100s std dev: 0.00011s window: 231  
average rate: 10.000  
min: 0.100s max: 0.100s std dev: 0.00011s window: 241  
average rate: 10.000  
min: 0.100s max: 0.100s std dev: 0.00012s window: 251
```

Example

- Step 4: start a listener node
Move to a different console and type

```
>roslaunch roscpp_tutorials listener
```

A terminal window titled 'luigi@ubuntu: ~' with four tabs. The first tab is 'roscore http://ubuntu:11311/'. The other three tabs are 'luigi@ubuntu: ~'. The terminal displays a series of log messages from a ROS listener node. Each message is on a new line and follows the format: '[INFO] [timestamp]: I heard: [hello world timestamp]'. The timestamps range from 1628083868.014052069 to 1628083868.914042106. The messages are: [INFO] [1628083868.014052069]: I heard: [hello world 13405], [INFO] [1628083868.114105275]: I heard: [hello world 13406], [INFO] [1628083868.214101641]: I heard: [hello world 13407], [INFO] [1628083868.314065579]: I heard: [hello world 13408], [INFO] [1628083868.414086186]: I heard: [hello world 13409], [INFO] [1628083868.514077303]: I heard: [hello world 13410], [INFO] [1628083868.614135561]: I heard: [hello world 13411], [INFO] [1628083868.714101058]: I heard: [hello world 13412], [INFO] [1628083868.814033440]: I heard: [hello world 13413], [INFO] [1628083868.914042106]: I heard: [hello world 13414].

```
luigi@ubuntu: ~  
roscore http://ubuntu:11311/ x luigi@ubuntu: ~ x luigi@ubuntu: ~ x luigi@ubuntu: ~  
[ INFO] [1628083868.014052069]: I heard: [hello world 13405]  
[ INFO] [1628083868.114105275]: I heard: [hello world 13406]  
[ INFO] [1628083868.214101641]: I heard: [hello world 13407]  
[ INFO] [1628083868.314065579]: I heard: [hello world 13408]  
[ INFO] [1628083868.414086186]: I heard: [hello world 13409]  
[ INFO] [1628083868.514077303]: I heard: [hello world 13410]  
[ INFO] [1628083868.614135561]: I heard: [hello world 13411]  
[ INFO] [1628083868.714101058]: I heard: [hello world 13412]  
[ INFO] [1628083868.814033440]: I heard: [hello world 13413]  
[ INFO] [1628083868.914042106]: I heard: [hello world 13414]  
[ ]
```

Example

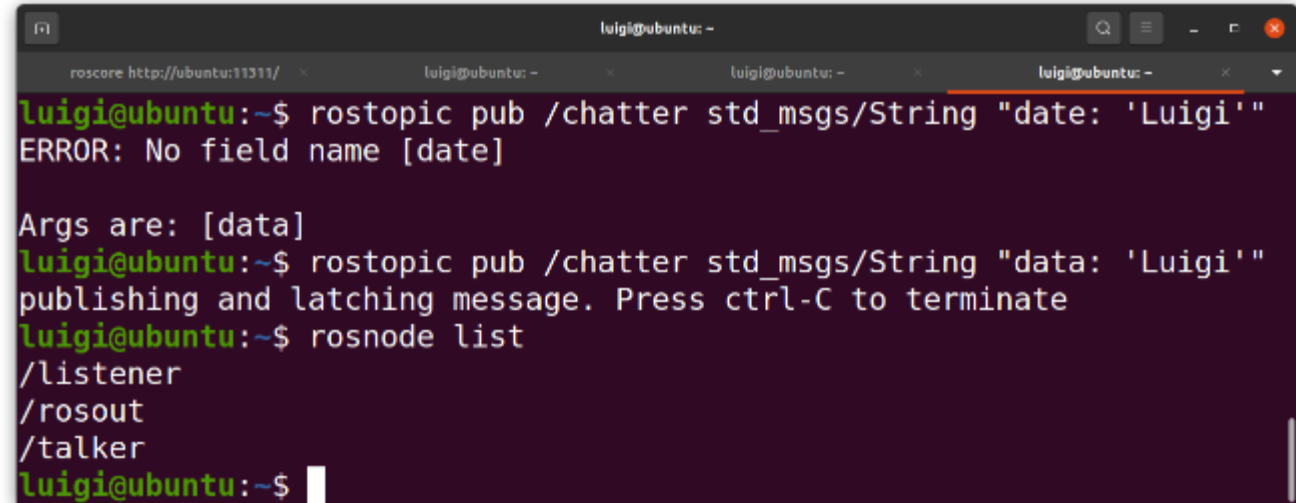
- Step 5: Analyse

Move to a different console and type

```
>roscnode list
```

Connection between the nodes
through chatter

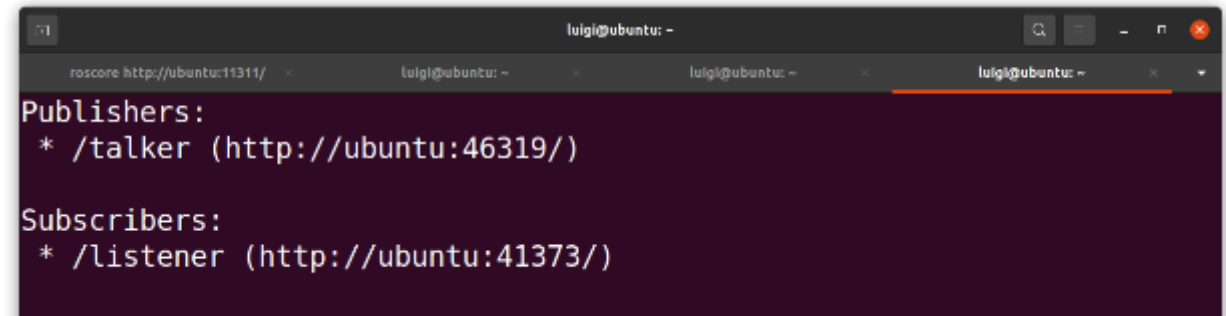
```
>roscnode info /chatter
```



A terminal window titled 'luigi@ubuntu: -' showing the execution of ROS commands. The first command is 'rostopic pub /chatter std_msgs/String "date: \'Luigi\'"', which results in an error: 'ERROR: No field name [date]'. The second command is 'rostopic pub /chatter std_msgs/String "data: \'Luigi\'"', which successfully publishes a message. The third command is 'roscnode list', which lists the active nodes: '/listener', '/rosout', and '/talker'.

```
luigi@ubuntu:~$ rostopic pub /chatter std_msgs/String "date: 'Luigi'"
ERROR: No field name [date]

Args are: [data]
luigi@ubuntu:~$ rostopic pub /chatter std_msgs/String "data: 'Luigi'"
publishing and latching message. Press ctrl-C to terminate
luigi@ubuntu:~$ roscnode list
/listener
/rosout
/talker
luigi@ubuntu:~$
```



A terminal window titled 'luigi@ubuntu: -' showing the output of the 'roscnode info /chatter' command. It displays the publishers and subscribers for the '/chatter' topic. The publisher is '/talker' at 'http://ubuntu:46319/' and the subscriber is '/listener' at 'http://ubuntu:41373/'.

```
luigi@ubuntu:~$ roscnode info /chatter
Publishers:
* /talker (http://ubuntu:46319/)

Subscribers:
* /listener (http://ubuntu:41373/)
```

Example

- Step 5: Publish a message from console

Move to the *talker* console and close the node by ctrl+c

Publish message through

```
>rostopic pub /chatter std_msgs/String "data: 'my message' "
```

```
luigi@ubuntu:~$ rostopic pub /chatter std_msgs/String "data: 'my message'"  
publishing and latching message. Press ctrl-C to terminate
```

Check the output on the listener console

```
[ INFO] [1628084786.014510482]: I heard: [hello world 22585]  
[ INFO] [1628084786.114515159]: I heard: [hello world 22586]  
[ INFO] [1628084786.214654978]: I heard: [hello world 22587]  
[ INFO] [1628084786.314529753]: I heard: [hello world 22588]  
[ INFO] [1628084792.822829555]: I heard: [my message]
```

Before moving on

- For the examples in the next slides, we will assume to have rqt and turtlesim installed, the second for illustration purposes
- If you have not done it already, take your time and type in your linux console

```
>sudo apt-get install ros-noetic-rqt ros-noetic-rqt-common-plugins ros-noetic-turtlesim
```

ROS Services

- Services are a different way for two nodes to communicate
- Services allow a node to send a request and receive a response
- The commands related to services are

<code>rosservice list</code>	print information about active services
<code>rosservice call</code>	call the service with the provided args
<code>rosservice type</code>	print service type
<code>rosservice find</code>	find services by service type
<code>rosservice uri</code>	print service ROSRPC uri

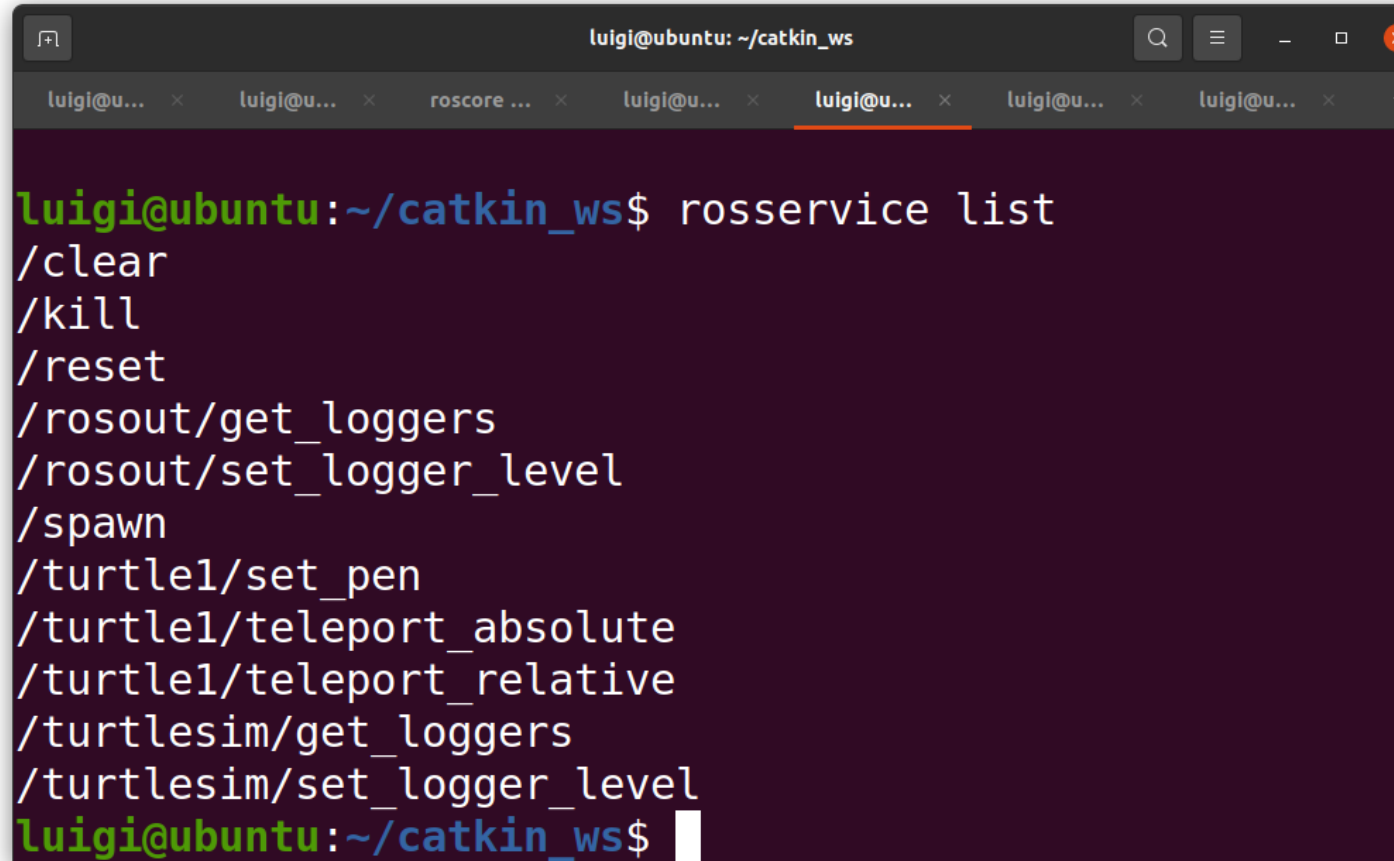
ROS Services

- If we try:

```
>roslaunch turtlesim  
turtlesim_node
```

```
>rosservice list
```

- We can see that the turtlesim node provides nine services: reset, clear, spawn, kill.

A terminal window titled 'luigi@ubuntu: ~/catkin_ws' showing the output of the 'rosservice list' command. The terminal has a dark background with light-colored text. The command 'rosservice list' is entered at the prompt, and the output lists nine services: /clear, /kill, /reset, /rosout/get_loggers, /rosout/set_logger_level, /spawn, /turtle1/set_pen, /turtle1/teleport_absolute, /turtle1/teleport_relative, /turtlesim/get_loggers, and /turtlesim/set_logger_level. The prompt is now ready for the next command.

```
luigi@ubuntu:~/catkin_ws$ rosservice list  
/clear  
/kill  
/reset  
/rosout/get_loggers  
/rosout/set_logger_level  
/spawn  
/turtle1/set_pen  
/turtle1/teleport_absolute  
/turtle1/teleport_relative  
/turtlesim/get_loggers  
/turtlesim/set_logger_level  
luigi@ubuntu:~/catkin_ws$
```

ROS Services

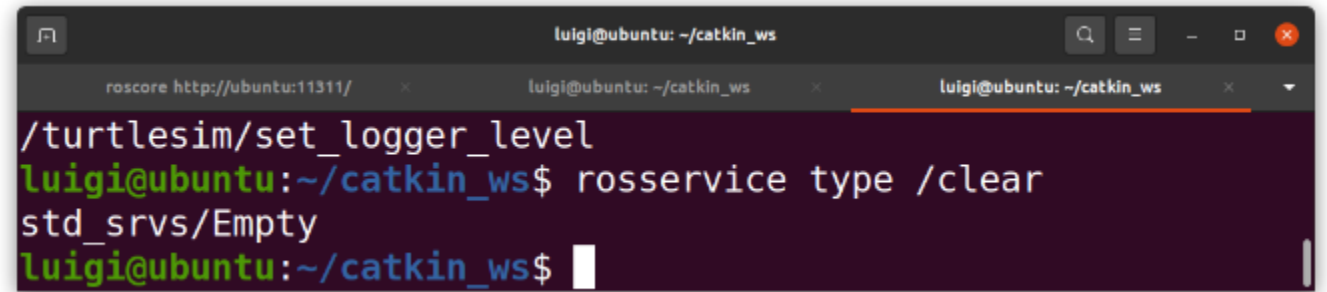
- Let us check the type of a services

```
>rosservice type /clear
```

- We can see that the service is empty (meaning that it takes no argument)
- The service can be invoked by

```
>rosservice call /clear
```

which clear the screen of the turtle simulator

A terminal window with a dark background and light text. The window title is 'luigi@ubuntu: ~/catkin_ws'. The terminal shows the command 'rosservice type /clear' being executed, with the output 'std_srvs/Empty'. The prompt 'luigi@ubuntu:~/catkin_ws\$' is visible at the end of the line.

```
luigi@ubuntu: ~/catkin_ws  
/turtlesim/set_logger_level  
luigi@ubuntu:~/catkin_ws$ rosservice type /clear  
std_srvs/Empty  
luigi@ubuntu:~/catkin_ws$
```

ROS Services

- Let us now try a service with arguments. Type

```
>rosservice type /spawn |  
rossrv show
```

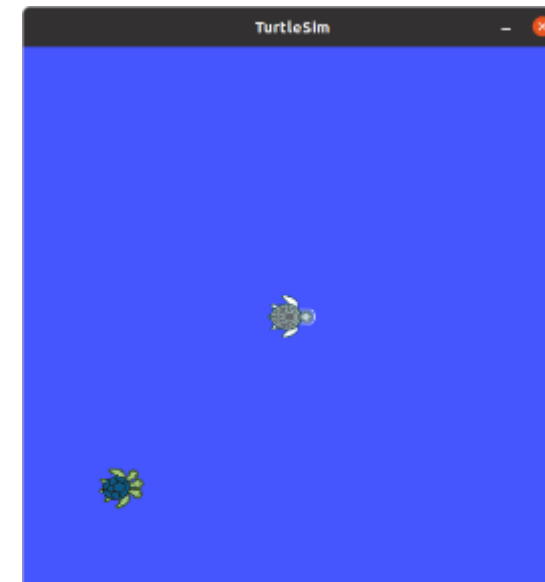
- Followed by

```
>rosservice call /spawn  
2 2 0.2 ""
```

- The outcome is to create a new turtle in a different position

```
luigi@ubuntu: ~/catkin_ws
rosservice type /spawn | rossrv show
float32 x
float32 y
float32 theta
string name
---
string name

luigi@ubuntu:~/catkin_ws$ rosservice call /spawn 2 2 0.2 ""
name: "turtle2"
luigi@ubuntu:~/catkin_ws$
```



ROS Parameters

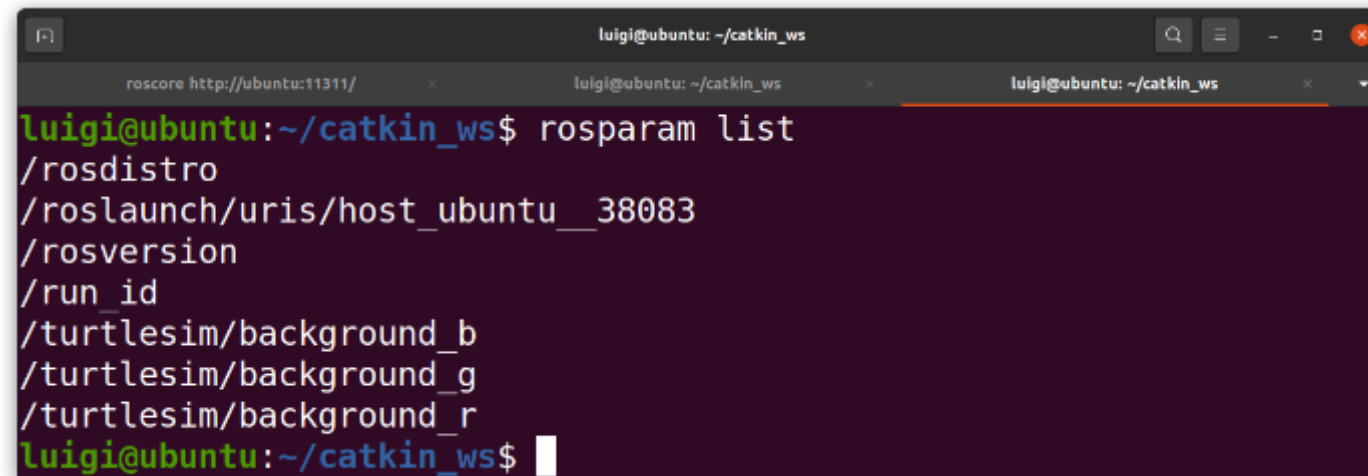
- The program `rosparam` allows us to create and manipulate data into the ROS parameter server
- The possible commands to manipulate parameters are

<code>rosparam set</code>	set parameter
<code>rosparam get</code>	get parameter
<code>rosparam load</code>	load parameters from file
<code>rosparam dump</code>	dump parameters to file
<code>rosparam delete</code>	delete parameter
<code>rosparam list</code>	list parameter names

ROS Parameters

- Example (with roscore open in another console)

```
>roslaunch turtlesim turtlesim_node  
>rosparam list
```

A terminal window titled 'luigi@ubuntu: ~/catkin_ws' showing the output of the 'rosparam list' command. The output lists several parameters: /roscore, /roslaunch, /rosversion, /run_id, /turtlesim/background_b, /turtlesim/background_g, and /turtlesim/background_r. The prompt 'luigi@ubuntu:~/catkin_ws\$' is visible at the bottom.

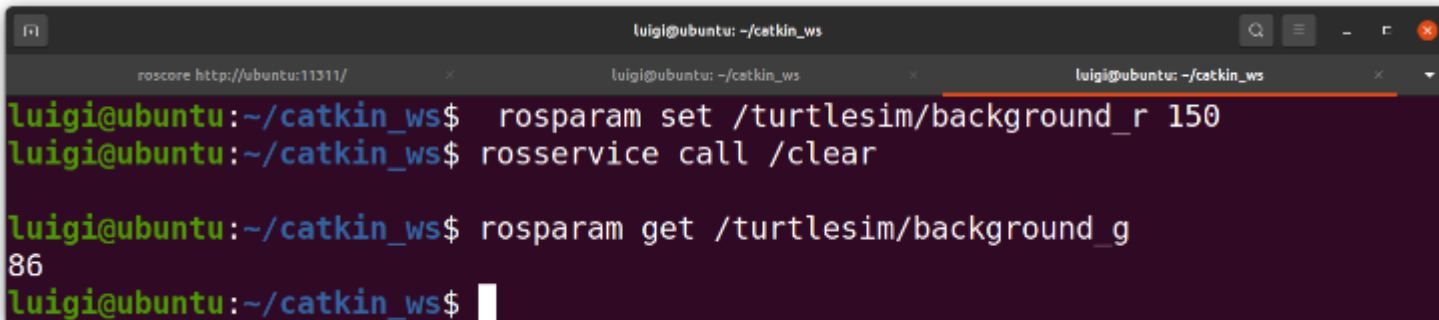
```
luigi@ubuntu:~/catkin_ws$ rosparam list  
/roscore  
/roslaunch/uris/host_ubuntu__38083  
/rosversion  
/run_id  
/turtlesim/background_b  
/turtlesim/background_g  
/turtlesim/background_r  
luigi@ubuntu:~/catkin_ws$
```

- We can see there is a param for background colour

ROS Parameters

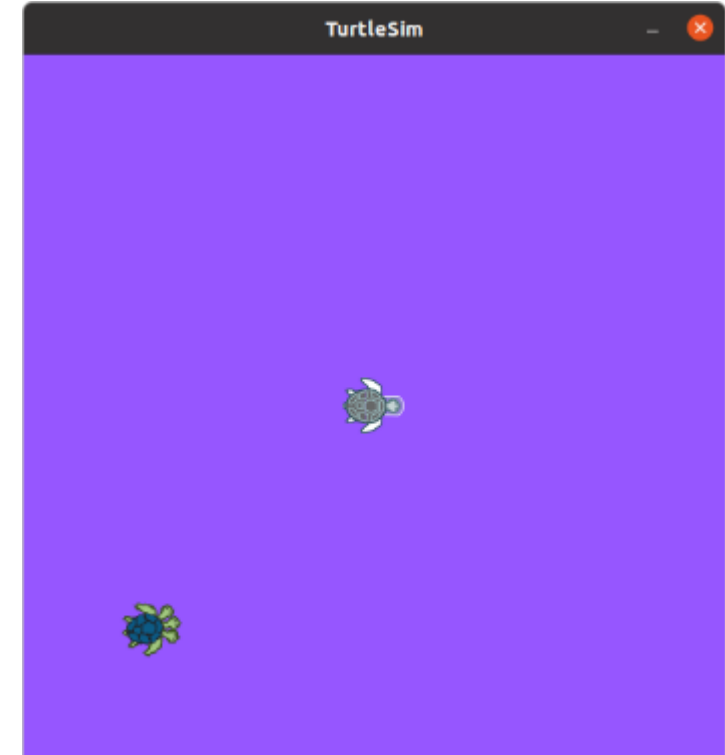
- Let us try

```
>rosparam set /turtlesim/background_r 150  
>rosservice call /clear  
>rosparam /turtlesim/background_r
```



A terminal window titled 'luigi@ubuntu: ~/catkin_ws' showing the execution of ROS commands. The prompt is 'luigi@ubuntu:~/catkin_ws\$'. The commands entered are: 'rosparam set /turtlesim/background_r 150', 'rosservice call /clear', and 'rosparam get /turtlesim/background_g'. The output for the last command is '86'. The terminal has a dark background with green and white text.

```
luigi@ubuntu:~/catkin_ws$ rosparam set /turtlesim/background_r 150  
luigi@ubuntu:~/catkin_ws$ rosservice call /clear  
luigi@ubuntu:~/catkin_ws$ rosparam get /turtlesim/background_g  
86  
luigi@ubuntu:~/catkin_ws$
```

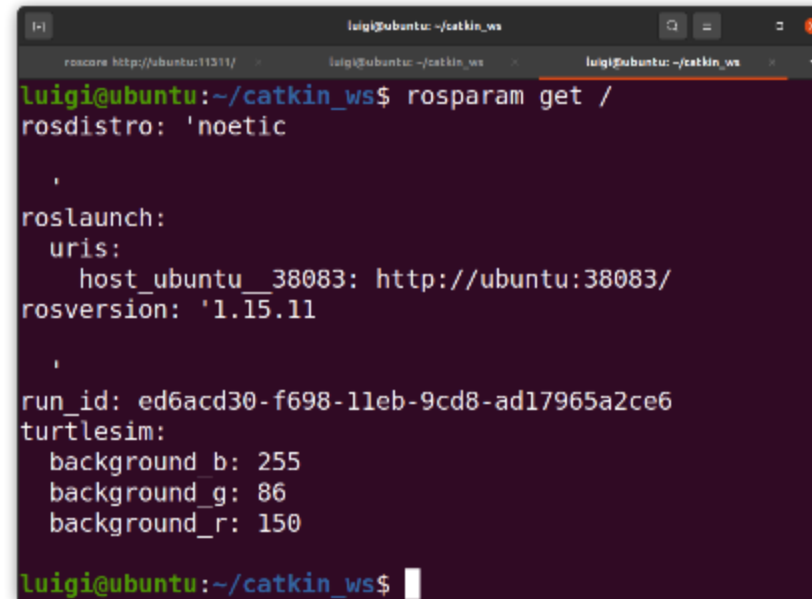


- We will change the colour of the simulation background and look up the green component

ROS Parameters

- You can print out the entire Parameter set by

```
>rosparam get /
```

A terminal window screenshot showing the command 'rosparam get /' being executed. The output lists ROS parameters: 'roscore' (http://ubuntu:11311/), 'roslaunch' (with 'uris' containing 'host_ubuntu_38083: http://ubuntu:38083/' and 'rosversion: '1.15.11''), 'run_id' (ed6acd30-f698-11eb-9cd8-ad17965a2ce6), and 'turtlesim' (with 'background_b: 255', 'background_g: 86', and 'background_r: 150'). The prompt 'luigi@ubuntu:~/catkin_ws\$' is visible at the bottom.

```
luigi@ubuntu:~/catkin_ws$ rosparam get /  
roscore: 'http://ubuntu:11311/'  
  
,  
roslaunch:  
  uris:  
    host_ubuntu_38083: http://ubuntu:38083/  
  rosversion: '1.15.11'  
  
,  
run_id: ed6acd30-f698-11eb-9cd8-ad17965a2ce6  
turtlesim:  
  background_b: 255  
  background_g: 86  
  background_r: 150  
  
luigi@ubuntu:~/catkin_ws$
```

- You can dump into file and retrieve by:

```
>rosparam dump [file_name]  
>rosparam load [file_name]
```

Debugging and launching nodes

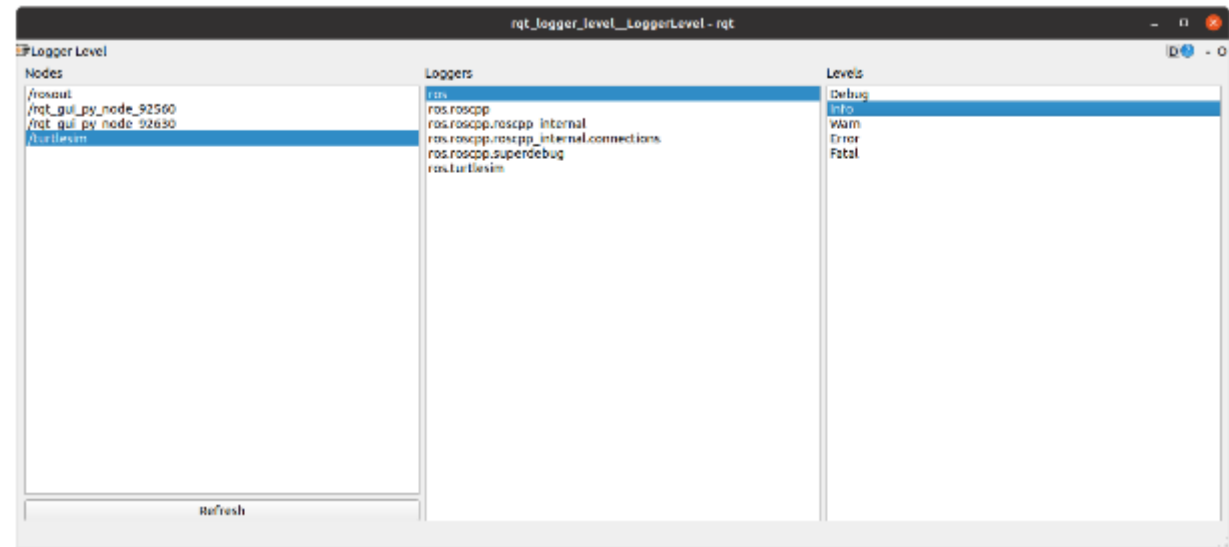
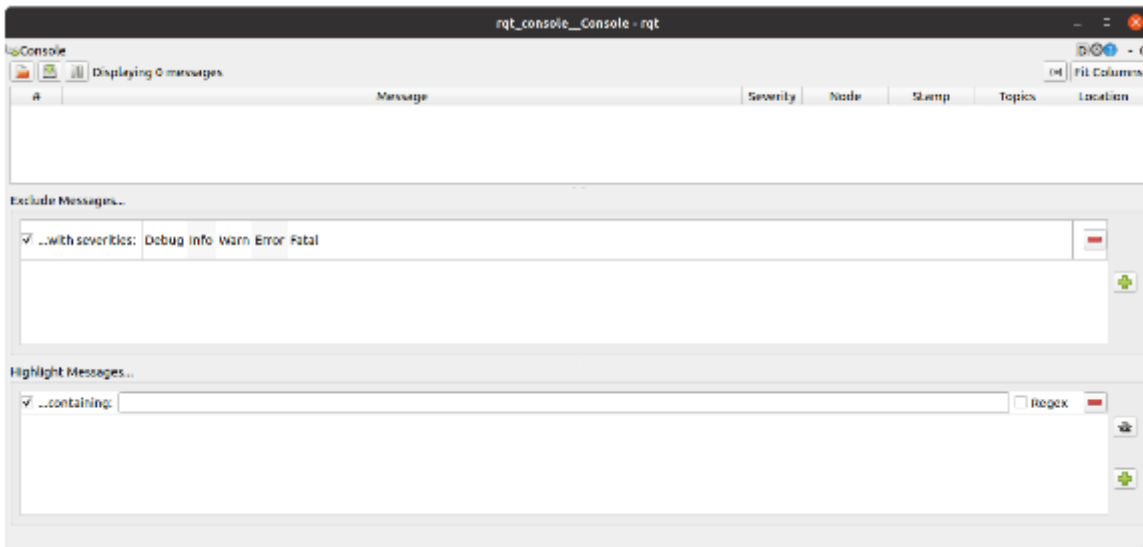
- Also for this part, we will assume to have `rqt` and `turtlesim` packages installed
- `rqt_console` attaches to the ROS's logging framework to display output from loggers
- The `rqt_logger_level` allows us to change the verbosity level (DEBUG, INFO, WARN, ERROR, FATAL)
- The four levels are in increasing level of criticality (DEBUG passes all, FATAL only the critical messages)

Using rqt_console

- Just type **in two different terminals** the following:

```
>roslaunch rqt_console rqt_console
```

```
>roslaunch rqt_logger_level rqt_logger_level
```



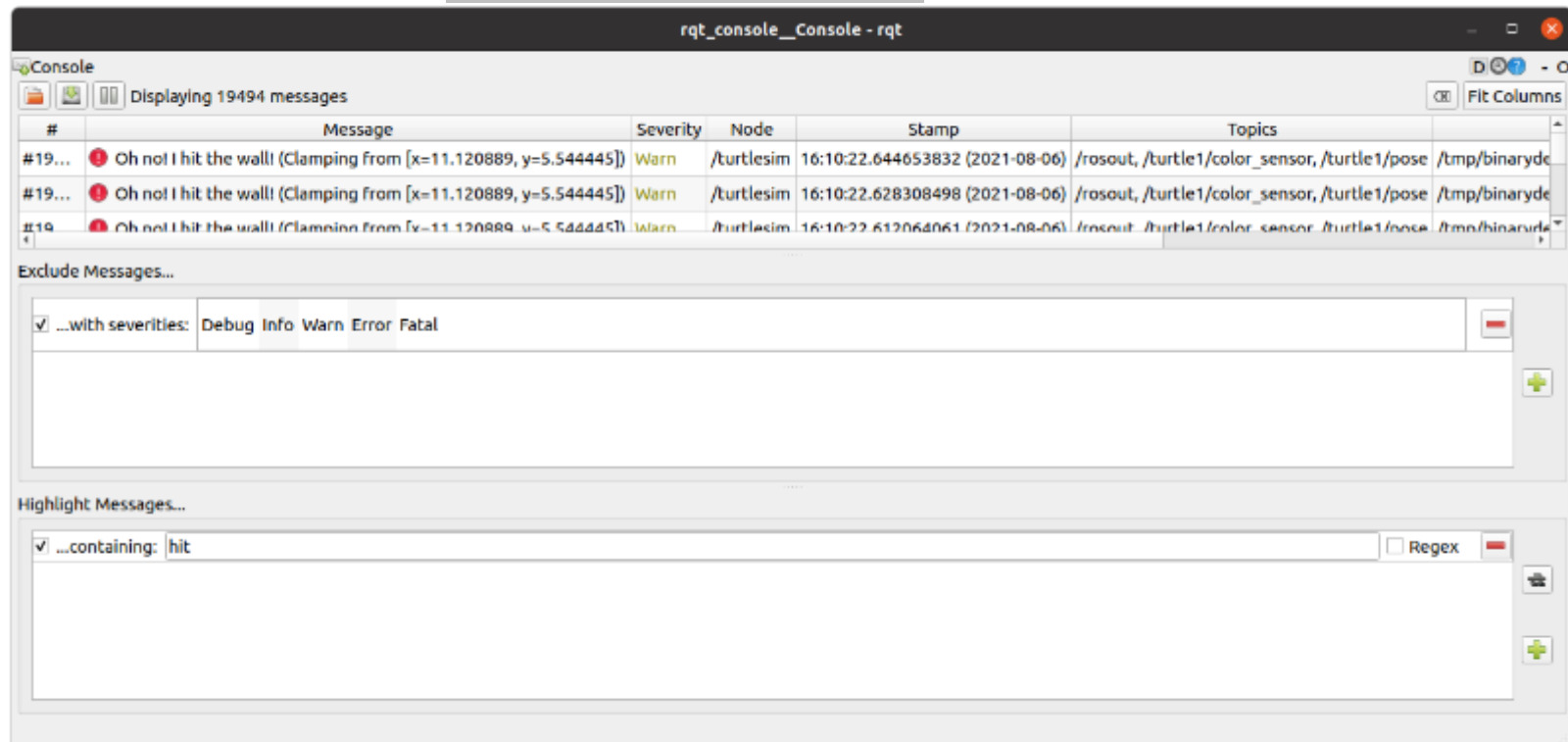
Using rqt_console

- Now let us active turtlesim and let us make the turtle walk until it crashes against the wall (use two different terminals)

```
>roslaunch turtlesim turtlesim_node
```

```
>rostopic pub /turtle1/cmd_vel  
geometry_msgs/Twist -r 1 -- '[2.0, 0.0, 0.0]'  
'[0.0, 0.0, 0.0]'
```

- On the Console we will start seeing



Using rqt_console

- If we now set the turtlesim logger level to Error, the message stream in the console will stop

